

Tecnológico de Monterrey
Inteligencia artificial avanzada para la ciencia de datos

Reporte

Predicción de jugadas de ajedrez a partir de la posición del tablero con redes neuronales convolucionales

---

Mauricio Olguín Sánchez
A01711522

Noviembre 2026

Índice

1. Introducción **1**
- 1.1. Problema y objetivo . . . 1
- 1.2. Alcance . . . 1

2. El ajedrez en este proyecto **1**
- 2.1. Tablero y piezas . . . 1
- 2.2. Reglas especiales . . . 1
- 2.3. Cómo se escriben las posiciones y las partidas . . . 2
- 2.4. Jugadas, *plies* y fases de la partida . . . 2
- 2.5. Nivel de los jugadores y ritmo de juego . . . 2

3. Antecedentes **2**

4. Datos y proceso de ETL **3**
- 4.1. Sobre la base de datos seleccionada . . . 3
- 4.2. Campos de cada partida . . . 3
- 4.3. Proceso de ETL . . . 3

5. Análisis exploratorio de los datos **6**
- 5.1. Nivel de los jugadores . . . 6
- 5.2. Momento de la partida y fases . . . 6
- 5.3. Jugadas legales y la línea base del azar . . . 7
- 5.4. Las jugadas: clases, piezas y capturas . . . 7
- 5.5. Posiciones que se repiten entre partidas . . . 8
- 5.6. Aperturas y partidas . . . 9

6. Metodología de evaluación **9**
- 6.1. Durante el entrenamiento . . . 9
- 6.2. Precisión *top-k* . . . 9
- 6.3. Conjuntos de entrenamiento y validación . . . 10
- 6.4. Líneas base . . . 10

7. Primer modelo **11**
- 7.1. Arquitectura . . . 11
- 7.2. Cómo llegué a esta arquitectura . . . 11
- 7.3. Entrenamiento final . . . 12
- 7.4. Resultados en prueba . . . 13
- 7.5. Fuerza de juego . . . 13
- 7.6. Análisis de errores . . . 14
- 7.7. Uso del modelo . . . 14

8. Códigos **14**

Referencias **16**

1. Introducción

1.1. Problema y objetivo

El objetivo de este proyecto es construir, con técnicas de *deep learning*, un modelo que juegue ajedrez: que, dada una posición del tablero, elija una jugada.
Para lograrlo, el modelo aprende de partidas reales. Planteé el problema como una **clasificación multiclase supervisada**, con el mismo flujo de una red convolucional para Fashion MNIST, solo que con otro tipo de imagen y muchas más clases. El tablero se trata como una imagen pequeña: una cuadrícula de 8×8 casillas con varios canales, donde cada canal indica dónde están las piezas de un tipo y color. La red recibe esa imagen y produce, con una capa *softmax*, una probabilidad para cada jugada posible. La etiqueta de cada ejemplo es la jugada que de verdad se jugó en esa posición, y el modelo juega eligiendo la jugada más probable entre las legales.
Como aprende de partidas entre jugadores humanos de nivel alto, se espera que juegue parecido a ellos y no como un motor de ajedrez. La primera forma de evaluarlo es medir qué tan seguido elige la jugada que hizo la persona.

1.2. Alcance

Una red que, a partir de una posición, elige la jugada. Se entrena con partidas de Lichess y se evalúa con la precisión *top-k*.

2. El ajedrez en este proyecto

Esta sección resume lo que hace falta saber de ajedrez para entender el resto del reporte: cómo es el tablero, qué reglas afectan la forma en que represento las posiciones y las jugadas, y cómo se escriben las partidas.

2.1. Tablero y piezas

El tablero tiene 8×8 casillas. Las columnas se nombran de la *a* a la *h* y las filas del 1 al 8, así que cada casilla tiene un nombre como e4. Las blancas empiezan en las filas 1 y 2 y las negras en las filas 7 y 8 (figura 1). Cada jugador tiene 16 piezas: un rey, una dama, dos torres, dos alfiles, dos caballos y ocho peones. La torre se mueve en línea recta, el alfil en diagonal, la dama en ambas direcciones, el rey una casilla en cualquier dirección y el caballo en forma de «L», saltando sobre las demás piezas. El peón avanza una casilla (dos en su primer movimiento) y captura en diagonal. Gana quien da **jaque mate**: atacar al rey rival de modo que no tenga forma de escapar.

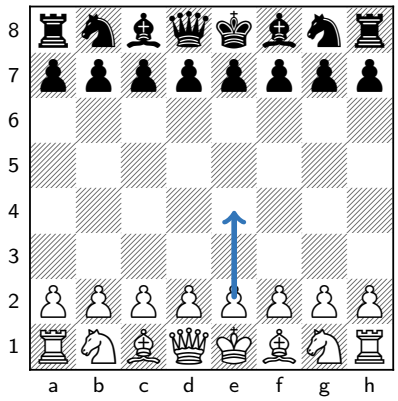


Figura 1: Posición inicial. La flecha muestra la jugada 1.e4.

Para el análisis uso el valor convencional de las piezas: peón 1, caballo 3, alfil 3, torre 5 y dama 9. El rey no tiene valor porque no se puede capturar.

2.2. Reglas especiales

Algunas reglas dependen de algo más que la posición de las piezas, y la representación del tablero (sección 4) guarda esa información adicional:

- **Enroque:** el rey se mueve dos casillas hacia una torre y la torre salta al otro lado del rey. Solo se permite si ninguno de los

dos se ha movido antes: son los **derechos de enroque** de cada jugador.

- **Captura al paso:** si un peón avanza dos casillas y queda junto a un peón rival, este puede capturarlo como si hubiera avanzado una sola, pero únicamente en la jugada inmediata.
- **Promoción:** el peón que llega a la última fila se convierte en otra pieza; casi siempre en dama.
- **Jaque y tablas:** un jugador no puede dejar a su rey en jaque. La partida termina en tablas por ahogado (no hay jugadas legales sin estar en jaque), por repetición, por la regla de los 50 movimientos, por material insuficiente o por acuerdo.

2.3. Cómo se escriben las posiciones y las partidas

- **SAN** (notación algebraica estándar): la forma habitual de escribir una jugada, con la letra de la pieza en inglés y la casilla de destino, por ejemplo `Nf3` (caballo a f3) o `e4` (peón a e4). Es la notación de los archivos de Lichess.
- **UCI**: la jugada como casilla de origen y de destino, por ejemplo `g1f3`. Es la base de mi etiqueta (sección 4).
- **FEN**: una línea de texto que describe una posición completa: piezas, a quién le toca, derechos de enroque, captura al paso y contadores. La posición inicial es:
  `rnbqkbnr/pppppppp/8/8/8/8/`
  `PPPPPPPP/RNBQKBNR w KQkq - 0 1.`
- **PGN**: el formato de archivo de las partidas. Tiene encabezados (jugadores, Elo, tipo de partida, resultado) seguidos de las jugadas en SAN, y admite comentarios entre llaves.

2.4. Jugadas, *plies* y fases de la partida

En ajedrez, una «jugada» numerada incluye el movimiento de las blancas y la respuesta de las negras. En este reporte cuento ***plies*** (medias jugadas): cada movimiento de un solo jugador es un *ply*, así que la posición inicial es el *ply* 0.
Una partida suele dividirse en tres fases: la **apertura**, en la que se desarrollan las piezas y muchos jugadores repiten secuencias conocidas; el **medio juego**; y el **final**, cuando quedan pocas piezas. No existe una frontera única entre fases; la que uso la defino en el análisis exploratorio (sección 5).

2.5. Nivel de los jugadores y ritmo de juego

Lichess califica a cada jugador con un sistema de la familia Elo, y los archivos guardan esa calificación en los encabezados `WhiteElo` y `BlackElo`. En estos sistemas lo que importa es la diferencia entre dos jugadores: mientras mayor es, más probable es que gane el de mayor calificación. En este reporte llamo «Elo» a esa calificación.
Lichess también clasifica cada partida por su ritmo, según el tiempo disponible: Bullet (el más rápido), Blitz, Rápidas y Clásicas.

3. Antecedentes

Antes de diseñar el modelo investigué otras implementaciones de este mismo problema y cómo evaluar un modelo que aprende de jugadas humanas. La métrica del artículo base [1], un R^2 sobre la diferencia de evaluación, no dice cuántas veces el modelo elige la jugada correcta.
McIlroy-Young et al. [5] entrenan Maia, una red residual, con unos 12 millones de partidas de Lichess por nivel, sin partidas Bullet, con división por partida y sin las primeras 10 jugadas. Eligiendo solo entre jugadas legales, acierta entre 46 % y más de 52 % de las jugadas humanas.

Arsalan y Soeparno [1], el artículo base, entrenan una CNN en Keras con 3,862 partidas de grandes maestros de Lichess y codifican cada pieza como un entero. Su red predice la diferencia de evaluación de la posición (regresión); el mejor modelo llega a $R^2 = 0{,}175$ y resuelve 4 de 28 problemas tácticos.
Soldera y Pedrini [6] usan una CNN con el tablero como $8\times8\times12$, un canal por pieza, y separan la salida en pieza y destino. Llegan a 68.5 % con un híbrido con Stockfish, contando como acierto cualquier jugada humana registrada para la posición.
Bertram et al. [2] trabajan con una variante de ajedrez con información imperfecta y representan el tablero como 12 planos de 8×8. Usar casi solo convoluciones rindió mejor que añadir capas densas, y preentrenar con autoencoders no mejoró el resultado.
En conjunto, estos trabajos muestran que predecir la jugada exacta es difícil: Maia, con una red grande y millones de partidas, acierta alrededor de la mitad. De ellos tomé varias decisiones: representar el tablero con un canal por tipo de pieza y color, en lugar de un entero por pieza que impone un orden sin sentido entre ellas; clasificar la jugada en lugar de predecir una evaluación; evaluar solo entre jugadas legales; dividir los datos por partida, y dejar fuera las partidas Bullet. A diferencia de Maia, no descarto las primeras jugadas: quiero que el modelo también sepa jugar la apertura, y reporto el resultado por fase de la partida. No comparo mi acierto con el de estos trabajos, que usan otros datos y criterios, sino con **líneas base calculadas con mis propios datos** (sección 6.4).

4. Datos y proceso de ETL

4.1. Sobre la base de datos seleccionada

Utilicé la *Lichess Open Database* [4], que publica cada mes todas las partidas jugadas en Lichess con licencia CC0. Cada archivo está en formato PGN comprimido con Zstandard (`.pgn.zst`).
Primero construí y probé el *pipeline* con el archivo de marzo de 2013, que pesa 22.5 MB comprimido y casi no tiene jugadores de nivel alto. El conjunto definitivo sale del mes completo más reciente al momento de procesarlo, septiembre de 2026 (`lichess_db_standard_rated_2026-09.pgn.zst`): pesa 29.2 GB comprimido y contiene 89,616,462 partidas.

4.2. Campos de cada partida

En el PGN, cada partida empieza con sus encabezados, una línea por campo con el nombre del campo y su valor entre comillas, y sigue con sus jugadas. La tabla 1 muestra los campos, con los de blancas y negras agrupados, y el uso que les doy.

4.3. Proceso de ETL

Para leer las partidas y generar las jugadas legales utilicé la biblioteca `chess` [3].

4.3.1. *Extraction*

El archivo se lee en *streaming*: se descomprime por bloques y se procesa una partida a la vez. Así nunca se guarda el PGN descomprimido en disco, que en un mes reciente ocuparía cientos de gigabytes, ni se carga completo en memoria.
Reconstruir el tablero jugada por jugada es el paso más lento, y en un mes reciente la gran mayoría de las partidas se descarta. Primero leo solo los encabezados de cada

Campo	Tipo	Descripción	Uso
Event	categórico	Ritmo (UltraBullet, Bullet, Blitz, Rapid, Classical o Correspondence) y si es de torneo.	Filtro de ritmo (descarta Bullet y UltraBullet) y comparación de ritmos en el análisis.
Site	texto	Enlace a la partida en Lichess.	–
Date, UTCDate, UTCTime	fecha y hora	Cuándo empezó la partida, en UTC.	–
Round	texto	Ronda del torneo; en Lichess siempre vale «-».	–
White, Black	texto	Nombres de usuario de las blancas y de las negras.	–
Result	categórico	1-0, 0-1, 1/2-1/2 o * (sin terminar).	–
WhiteElo, BlackElo	numérico	Elo de cada jugador al empezar la partida.	Filtro de nivel; se guarda el del jugador que mueve en cada posición.
WhiteRatingDiff, BlackRatingDiff	numérico	Puntos de Elo que ganó o perdió cada jugador con la partida.	–
WhiteTitle, BlackTitle	categórico	Título del jugador, si tiene: GM, IM, FM, NM, LM o BOT (cuentas de programas de ajedrez).	Filtro de cuentas BOT.
ECO	categórico	Código de apertura de la *Encyclopaedia of Chess Openings* (A00 a E99).	Análisis y acierto por apertura.
Opening	categórico	Nombre de la apertura, por ejemplo «Sicilian Defense: Smith-Morra Gambit».	Análisis y acierto por apertura.
TimeControl	texto	Segundos iniciales más segundos que se suman por jugada, por ejemplo 180+2.	–
Termination	categórico	Cómo terminó: Normal (mate, rendición o tablas), Time forfeit (sin tiempo) y, en pocos casos, otras causas.	Análisis de cómo terminan las partidas.
Jugadas	texto	Jugadas numeradas en SAN, cada una con un comentario con el tiempo restante de quien la jugó y, en una de cada diez partidas, la evaluación de un motor.	Reconstrucción de cada posición; los comentarios se ignoran.

Cuadro 1: Campos de cada partida del PGN y uso que les doy. Los que no se usan llevan «–».

partida y aplico con ellos los filtros; las jugadas solo se analizan en las partidas que los pasan. Filtrar el mes completo toma unos 10 minutos, y transformar las partidas que los pasan, unos 40.

4.3.2. *Transforming*

Filtros Apliqué los filtros en este orden (tabla 2):

- **Ritmo:** descarté las partidas Bullet y UltraBullet, porque con tan poco tiempo las jugadas son más impulsivas y menos representativas.
- **Cuentas BOT:** descarté las partidas en las que juega alguna cuenta BOT, que son programas de ajedrez. Este filtro lo agregué a partir del análisis exploratorio (sección 5).

- **Nivel:** conservé solo las partidas en las que ambos jugadores tienen un Elo de 2,200 o más.

Pasan los tres filtros 1,507,816 partidas; 2,023 no tienen jugadas y quedan 1,505,793. Las transformo por bloques de 100 mil partidas para no agotar la memoria, y los uno al final.

Paso	Partidas
Partidas leídas	89,616,462
Bullet o UltraBullet	32,517,004
Con alguna cuenta BOT	325,060
Algún jugador con Elo menor a 2,200	55,266,582
Sin jugadas	2,023
Partidas utilizadas	**1,505,793**

Cuadro 2: Partidas descartadas en cada filtro del mes de septiembre de 2026, en el orden en que se aplican. Las partidas BOT que también son Bullet se cuentan como Bullet.

El tablero visto desde quien mueve Para que la red no tenga que aprender dos veces lo mismo (una vez para las blancas y otra para las negras), represento siempre la posición **desde el punto de vista del jugador que mueve**. Cuando mueven las negras, volteo el tablero verticalmente e intercambio los colores, de modo que el jugador que mueve siempre aparece abajo.
Voltear el tablero de izquierda a derecha, en cambio, **no** es válido: el rey y la dama no son simétricos y el enroque cambiaría de sentido. La única simetría válida es voltear verticalmente e intercambiar los colores.

Representación del tablero La red recibe cada posición como una «imagen» de 8×8 con 17 canales binarios (tabla 3): uno por cada tipo de pieza y color, más los derechos de enroque y la captura al paso.

Canales	Contenido
0–5	Peones, caballos, alfiles, torres, dama y rey del jugador que mueve
6–11	Las mismas piezas del rival
12–15	Derechos de enroque (corto y largo de cada jugador), canal completo en 1 o en 0
16	Casilla de captura al paso, si existe

Cuadro 3: Canales de la representación *one-hot* del tablero.

Representación de la jugada La etiqueta es la jugada que se jugó, codificada como un número entre 0 y 4,095:

$$\text{clase} = 64 \times \text{origen} + \text{destino},$$

donde origen y destino son las casillas numeradas de 0 (a1) a 63 (h8), recorriendo cada fila de izquierda a derecha, ya en la orientación de quien mueve. Por ejemplo, 1.e4 (de e2, casilla 12, a e4, casilla 28) es la clase $64 \times 12 + 28 = 796$. Con esta orientación, 1.e4 de las blancas y 1. . . e5 de las negras tienen la misma clase. El enroque queda representado como un movimiento del rey de dos casillas. Las promociones se representan como promoción a dama; las 468 posiciones en las que se jugó una promoción a otra pieza se omitieron.

Jugadas legales Para cada posición guardo también la lista de sus jugadas legales y el valor de la pieza capturada por cada una, para descartar las jugadas ilegales al evaluar el modelo y calcular las líneas base.

Muestreo de posiciones Como las posiciones consecutivas de una misma partida se parecen mucho entre sí, tomé de cada una 12 al azar (o todas, si la partida es más corta), con semilla fija. Después de omitir las subpromociones, el resultado fue de **18,022,575 posiciones**.

4.3.3. *Loading*

El conjunto procesado se guarda en disco para no repetir la transformación cada vez

que se entrena. Cada fila es una posición, con su tablero (junto con los derechos de enroque y la captura al paso), la jugada que se jugó, la partida de la que sale, su *ply*, el Elo del jugador que mueve y la lista de sus jugadas legales. Los canales de la tabla 3 se construyen a partir del tablero, los derechos de enroque y la captura al paso al momento de entrenar.

La división en entrenamiento, validación y prueba se hace **por partida y no por posición**. Si se dividiera por posición, posiciones casi idénticas de la misma partida quedarían en entrenamiento y en prueba, y las métricas saldrían infladas por una fuga de datos. Mezclé las partidas con la semilla 42 y asigné 80 %, 10 % y 10 % de ellas a cada conjunto (tabla 4).

Conjunto	Partidas	Posiciones
Train	1,204,634	14,418,292
Validation	150,579	1,802,132
Test	150,580	1,802,151

Cuadro 4: División por partida con semilla 42.

El conjunto de *test* no interviene en ninguna decisión: elegí la arquitectura y los hiperparámetros con *validation*, y usé *test* una sola vez, para evaluar el modelo final.

4.3.4. Validación del proceso

Escribí pruebas automáticas antes de procesar los datos, porque un error en la codificación invalidaría todos los resultados. Entre otras cosas, comprueban que cada jugada se pueda codificar y decodificar sin pérdida (incluidos el enroque, la captura al paso y la promoción), que una posición con negras y su versión espejada tengan la misma codificación, que las etiquetas coincidan con las jugadas reales de partidas de ejemplo, que cada filtro descarte las partidas correctas y que la división no repita partidas entre conjuntos.

5. Análisis exploratorio de los datos

Antes de entrenar revisé el conjunto procesado para conocer a los jugadores, el momento de la partida en que ocurre cada posición, las jugadas que se eligen y qué tanto se repiten las posiciones entre partidas.

5.1. Nivel de los jugadores

El Elo del jugador que mueve va de 2,213 a 2,584 en el 90 % central de las posiciones, con mediana de 2,326 y máximo de 3,064 (figura 2). El 29.1 % de las posiciones corresponde a jugadores con 2,400 o más y el 1.2 %, a jugadores con 2,700 o más.

En la primera versión de este histograma apareció un segundo pico alrededor de 3,000. Al revisar los encabezados encontré que eran cuentas BOT. En las primeras 28 horas del mes, el 5.7 % de las partidas que no son Bullet entre jugadores de 2,200 o más tenía al menos una cuenta BOT, casi siempre una contra otra, con un Elo mediano de 2,718.

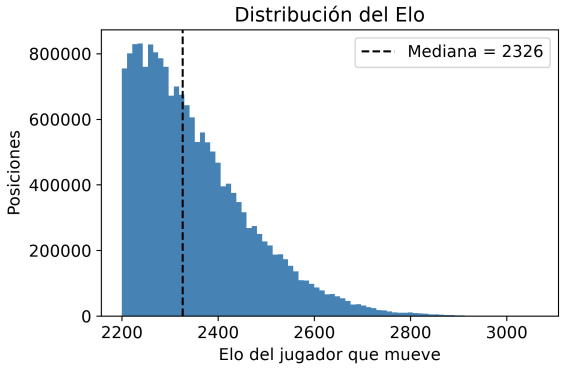


Figura 2: Elo del jugador que mueve, ya sin las partidas de cuentas BOT.

5.2. Momento de la partida y fases

Las posiciones se concentran en los primeros *plies* y su frecuencia baja conforme avanza la partida, porque muchas partidas terminan antes (figura 3); el *ply* promedio es 39.1.

Para analizar los resultados por fase definí tres grupos. El **final** son las posiciones con 12 piezas o menos, contando a los reyes; la **apertura**, el resto de las posiciones antes del *ply* 20 (diez jugadas de cada bando); y el **medio juego**, todo lo demás. Con esta definición, el 30.7 % de las posiciones es de apertura, el 60.1 % de medio juego y el 9.2 % de final (figura 4). Como el final tiene pocas posiciones, sus resultados tendrán más incertidumbre.

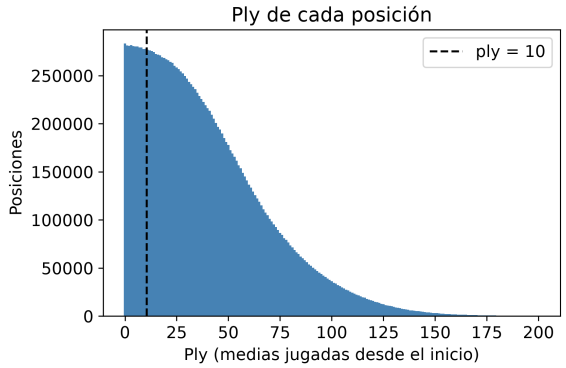


Figura 3: *Ply* en el que ocurre cada posición muestreada.

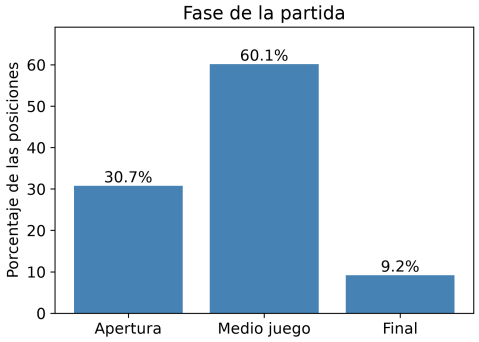


Figura 4: Posiciones por fase de la partida.

5.3. Jugadas legales y la línea base del azar

Cada posición tiene en promedio **31.0 jugadas legales**, con un mínimo de 1 y un máximo de 113 (figura 5). El pico en 20 corresponde a la posición inicial y a otras posiciones de apertura muy parecidas. El 0.6 % de las posiciones tiene una sola jugada legal y el 4.0 %, cuatro o menos.

Si se elige una jugada legal al azar, se acierta el **5.4 %** de las veces en promedio. Es más que $1/31{,}0 \approx 3{,}2\,\%$ porque en las posiciones con pocas opciones, como las de jaque, el azar acierta mucho más.

5.4. Las jugadas: clases, piezas y capturas

Solo aparecen 1,792 de las 4,096 clases posibles, porque muchas combinaciones de origen y destino no corresponden al movimiento de ninguna pieza. Además, las clases que sí aparecen están muy desbalanceadas. La figura 6 ordena las jugadas de la más a la menos común y muestra qué porcentaje del total se acumula: las 74 jugadas más comunes ya son la mitad de todas las jugadas del conjunto, 348 son el 80 % y 990 el 95 %. Las otras 802 juntas apenas suman el 5 % restante.

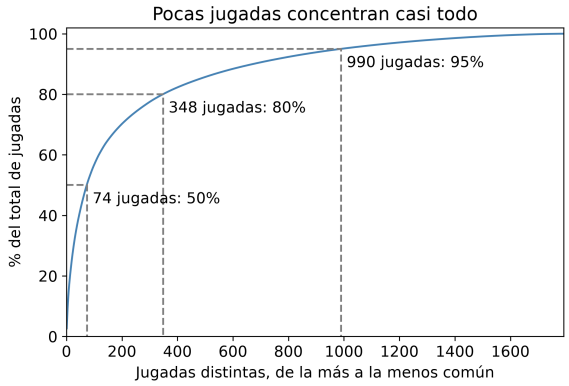


Figura 6: Porcentaje acumulado de jugadas al sumar las clases de la más a la menos común. Las líneas punteadas marcan cuántas clases hacen falta para cubrir el 50 %, el 80 % y el 95 % de las jugadas.

La pieza que más se mueve es el peón (29.2 %), seguida del caballo (18.4 %) y el alfil (15.6 %); la torre, la dama y el rey se mueven entre el 11 % y el 14 % de las veces (figura 7).

El 22.3 % de las jugadas son capturas (figura 8). En el 75.7 % de las posiciones hay al menos una captura disponible, pero solo en el 25.0 % de ellas se juega la captura de mayor valor.

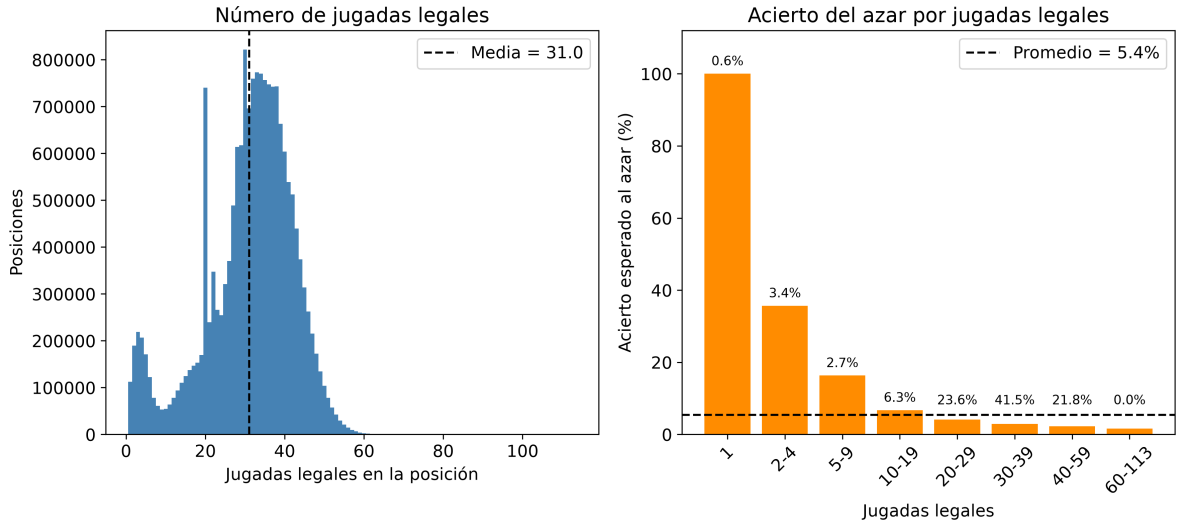


Figura 5: Izquierda: número de jugadas legales por posición. Derecha: acierto esperado al elegir una jugada legal al azar, por intervalo; encima de cada barra, el porcentaje de posiciones del intervalo.

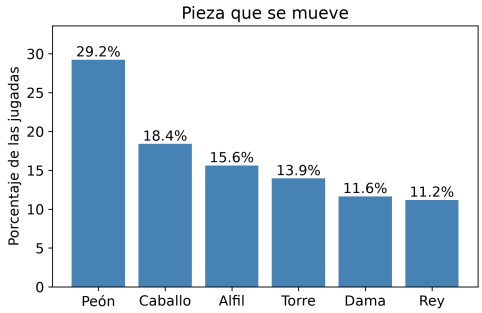


Figura 7: Tipo de pieza que se mueve en cada jugada.

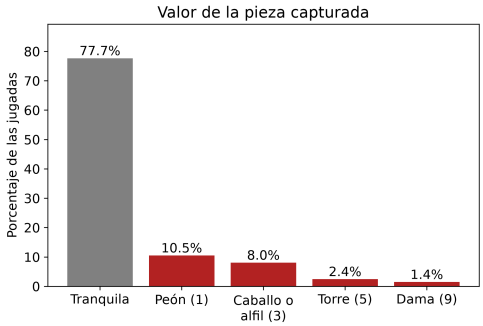


Figura 8: Jugadas tranquilas y capturas, según el valor de la pieza capturada.

5.5. Posiciones que se repiten entre partidas

Como incluí las aperturas, conté cuántas posiciones aparecen idénticas (mismas piezas, derechos de enroque y captura al paso) en dos o más partidas distintas. Se repiten porque muchos jugadores juegan las mismas aperturas, y en ellas el modelo puede acertar de memoria.

El 24.5 % de las posiciones aparece en al menos dos partidas. Casi todas son de apertura: ahí se repite el 75.6 % de las posiciones, mientras que en el resto de la partida solo el 1.9 %. Por *ply*, la posición inicial se repite siempre, en el *ply* 4 el 99.4 %, en el *ply* 10 el 82.9 %, en el *ply* 16 el 42.7 % y desde el *ply* 28 menos del 2 % (figura 9).

En esas posiciones repetidas, la jugada más frecuente se elige el 62.3 % de las veces. En una misma posición distintos jugadores eligen jugadas distintas, y ningún modelo que solo vea el tablero puede acertar todas: en estas posiciones, el acierto *top-1* tiene un techo cercano al 62 %. En la posición inicial el techo es todavía más bajo, del 48.1 %, porque los jugadores reparten su primera jugada entre varias opciones (sección 5.6).

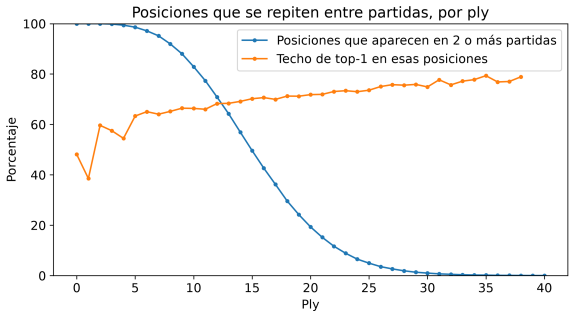


Figura 9: Por *ply*: porcentaje de posiciones que aparecen en dos o más partidas y techo del acierto *top-1* en esas posiciones. Solo se grafican los *plies* con al menos 100 posiciones.

5.6. Aperturas y partidas

Las secciones anteriores describen posiciones; esta describe las partidas de las que salen.

Aperturas. Lichess anota en cada partida el nombre de su apertura, con la forma «familia: variante», por ejemplo «Sicilian Defense: Najdorf Variation». La Defensa Siciliana sola está en el 17.4 % de las partidas; le siguen la Francesa (6.7 %), la Caro-Kann (6.1 %) y la Inglesa (6.1 %). Bastan 8 familias para cubrir la mitad de las partidas, 25 para el 80 % y 55 para el 95 % (figura 10).

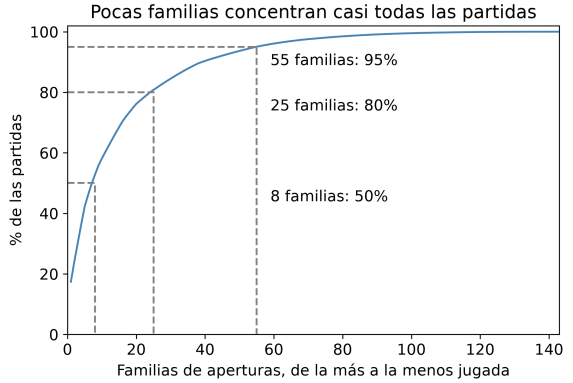


Figura 10: Porcentaje acumulado de las partidas al sumar las familias de aperturas, de la más a la menos jugada.

Primera jugada. Las blancas abren con 1.e4 en el 47.6 % de las partidas y con 1.d4 en el 32.5 %. Contra 1.e4, las negras responden sobre todo 1. . . c5, la Siciliana (36.2 %); contra 1.d4, 1. . . Nf6 (46.8 %) (figura 11).

Ritmos y duración. El conjunto es casi todo Blitz: el 92.2 % de las partidas; las Rápidas son el 7.4 % y las Clásicas y por correspondencia juntas, el 0.4 %. En el 23.3 % de las partidas a un jugador se le acabó el tiempo. Una partida dura en mediana 74 *plies* (37 jugadas por bando), y el 90 % dura entre 32 y 140 *plies*.

6. Metodología de evaluación

6.1. Durante el entrenamiento

La red se entrena con la entropía cruzada categórica (*sparse categorical crossentropy*), la pérdida para clasificación cuando las etiquetas son enteros. En cada época registro la pérdida y la precisión en entrenamiento y validación. Con *ModelCheckpoint* guardo el modelo de menor pérdida en validación, y con *EarlyStopping* detengo el entrenamiento cuando esa pérdida no mejora en dos épocas seguidas.

6.2. Precisión *top-k*

Para cada posición el modelo da una probabilidad a cada una de las 4,096 jugadas posibles. Primero descarto las jugadas ilegales, porque en una partida nunca podrían elegirse, y ordeno las legales de la más a la menos probable. Después busco en qué lugar de esa lista quedó la jugada que realmente se jugó. La precisión *top-k* es el porcentaje de posiciones en las que la jugada real quedó entre las k primeras; con $k = 1$ es la exactitud usual de un clasificador.

Por ejemplo, si en la posición inicial el modelo ordena 1.e4 (40 %), 1.d4 (30 %) y 1. Nf3 (10 %), y el jugador eligió 1.d4, la jugada real quedó en segundo lugar: la posición cuenta como fallo en *top-1* y como acierto en *top-3* y *top-5*.

Elegí esta métrica porque es la que usan los trabajos que, como este, buscan predecir la jugada de una persona. McIlroy-Young et

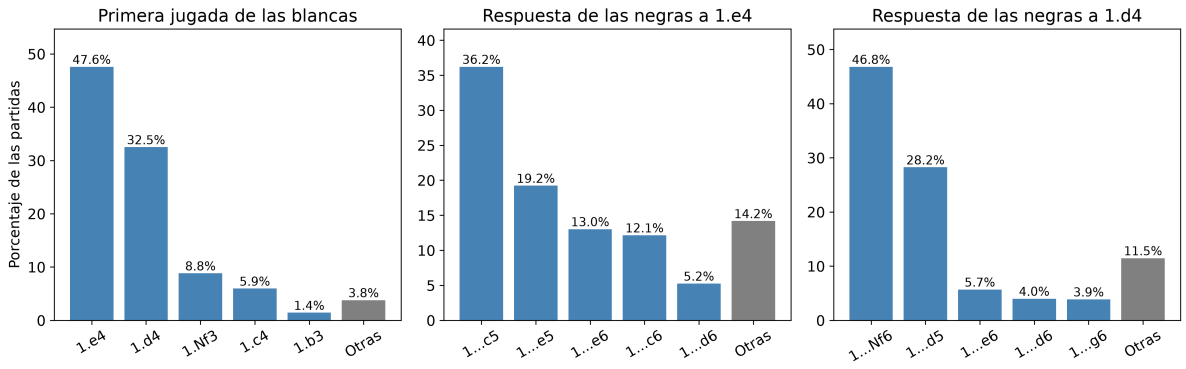


Figura 11: Primera jugada de las blancas y respuesta de las negras a 1.e4 y a 1.d4, en porcentaje de las partidas que empiezan así.

al. [5] miden la precisión de coincidencia con la jugada humana (*move-matching accuracy*), que es el *top-1*. Ellos mismos señalan su límite: como jugadores distintos eligen jugadas distintas en la misma posición, ningún modelo puede coincidir con todos (sección 5). También reporto *top-3* y *top-5*, como hacen Soldera y Pedrini [6] con su lista ordenada de jugadas candidatas (ellos llegan a *top-3*): si la jugada humana queda en segundo o tercer lugar, el modelo la consideró aunque no la eligiera.

También mido qué porcentaje de sus primeras opciones es ilegal, para saber si aprendió las reglas. Las líneas base de la sección 6.4 se evalúan exactamente igual: cada una ordena las jugadas legales con su propia regla y se mide en qué lugar quedó la jugada real. La métrica principal se calcula sobre **todas** las posiciones, incluidas las de apertura. Además la desgloso por fase (apertura, medio juego y final).

6.3. Conjuntos de entrenamiento y validación

El 24.1 % de las posiciones de validación aparece idéntica en entrenamiento. Casi todas son de apertura: ahí está repetido el 74.5 % de las posiciones, contra el 1.7 % en el medio juego y el 1.4 % en el final. No es una fuga de datos: son posiciones de partidas distintas que coinciden porque los jugadores repiten las mismas aperturas. En ellas se puede acertar solo con memoria, que es lo que mide la línea base del «libro».

6.4. Líneas base

Todas las métricas se comparan con cuatro líneas base calculadas con los mismos datos:

1. una jugada legal al azar;
2. la jugada legal más frecuente en el conjunto de entrenamiento;
3. la captura de mayor valor y, si no hay capturas, una jugada legal al azar;
4. un «libro»: si la posición exacta aparece en entrenamiento, la jugada que más se jugó en ella y, si no aparece, la jugada legal más frecuente.

Cuando dos jugadas empatan, por ejemplo dos capturas del mismo valor, el orden entre ellas se decide al azar. La tabla 5 muestra el acierto de cada línea base en validación.

El «libro» es la mejor línea base en el conjunto completo, pero casi todo su acierto viene de la apertura; fuera de ella las posiciones casi nunca están en entrenamiento y apenas supera al azar.

Capturar la pieza más valiosa es la mejor regla en el medio juego y en el final: aunque los jugadores no suelen elegirla, ninguna de las otras reglas se acerca a ella fuera de la apertura.

El modelo debe superar al «libro» en la

	Todas las posiciones			*Top-1* por fase		
Línea base	*Top-1*	*Top-3*	*Top-5*	Apertura	Medio juego	Final
Azar	5.4	14.4	21.7	3.6	5.3	11.9
Más frecuente	11.0	22.9	33.0	18.8	6.3	15.1
Captura de mayor valor	15.6	29.0	36.1	10.3	**17.6**	**20.7**
Libro	**20.1**	**31.0**	**38.9**	**46.8**	7.1	15.6
Posiciones			1,802,132	554,168	1,083,669	164,295

Cuadro 5: Acierto de las líneas base en validación (%). En negritas, la mejor de cada columna.

apertura, donde se puede acertar de memoria, y a la captura de mayor valor en el resto de la partida, donde tiene que entender la posición.

7. Primer modelo

7.1. Arquitectura

El modelo es una red convolucional sencilla, implementada y entrenada con TensorFlow/Keras (figura 12 y tabla 6). Recibe el tablero con los 17 canales de la tabla 3 y devuelve una probabilidad para cada una de las 4,096 jugadas posibles. Las dos capas convolucionales usan filtros de 3×3 con relleno (*padding*) para conservar el tamaño de 8×8. No usé *pooling*, porque en un tablero tan pequeño cada reducción borra casillas que importan. Casi todos los parámetros están en las dos capas densas.

Cada filtro de la primera capa mide $3 \times 3 \times 17$: recorre el tablero mirando a la vez las 3×3 casillas de alrededor en los 17 canales. Las piezas propias y rivales están en la profundidad de cada filtro y no determinan cuántos filtros hay; el número de filtros es cuántos patrones distintos puede aprender la capa. Usé 64 en la primera capa y 128 en la segunda porque es la configuración inicial habitual para este tipo de redes: más filtros que canales de entrada, y el doble en la capa siguiente, que combina esos patrones en zonas más grandes del tablero.

Después de la primera capa densa hay un *Dropout* con tasa de 0.5: en cada paso del entrenamiento apaga al azar la mitad de las salidas de esa capa, para que la red no dependa de unas cuantas neuronas para recordar posiciones concretas. Al predecir no apaga nada.

Capa	Salida	Parámetros
Conv. 3×3, 64 filtros	$8 \times 8 \times 64$	9,856
Conv. 3×3, 128 filtros	$8 \times 8 \times 128$	73,856
Flatten	8,192	0
Densa de 2,048	2,048	16,779,264
Dropout (0.5)	2,048	0
Densa de 4,096	4,096	8,392,704
Total		25,255,680

Cuadro 6: Arquitectura del modelo. Las capas convolucionales y la primera densa usan ReLU; la última, softmax.

7.2. Cómo llegué a esta arquitectura

Red inicial. Empecé con la misma red sin *Dropout* y con una primera capa densa de 1,024 neuronas (12.7 millones de parámetros), entrenada con una muestra al azar de una de cada cuatro partidas: 3.6 millones de posiciones de entrenamiento y 450,591 de validación. Entrené en lotes de 512, con Adam y durante 10 épocas. La pérdida en validación baja hasta la tercera época (2.17) y después sube, mientras que en entrenamiento sigue bajando (figura 13): desde la cuarta época la red memoriza las posiciones de entrenamiento en lugar de aprender algo que sirva para posiciones nuevas. En la décima época acierta el 57.8 % en entrenamiento y solo el 36.3 % en validación. Con la red de la tercera época, la que guardó *ModelCheckpoint*, el acierto *top-1* en validación fue de 39.2 %, y el 2.6 % de sus primeras opciones era ilegal.

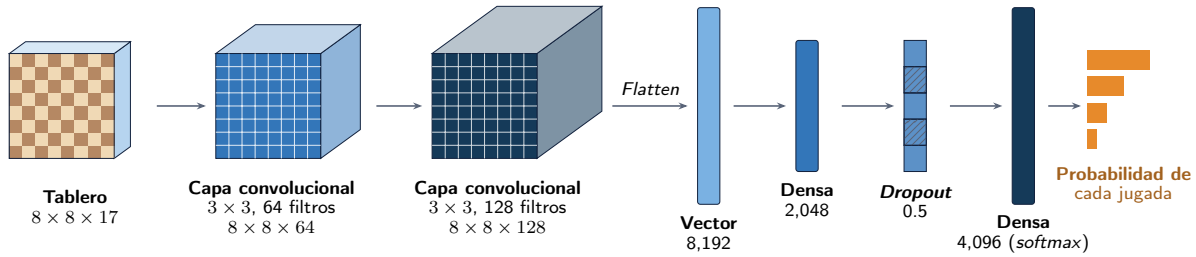


Figura 12: Arquitectura del modelo: del tablero a la probabilidad de cada una de las 4,096 jugadas. Bajo cada capa, el tamaño de su salida.

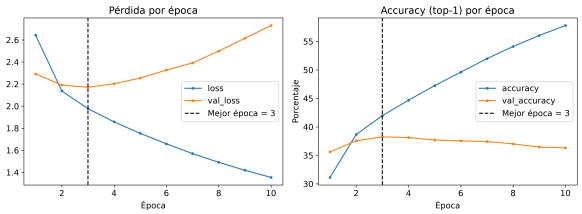


Figura 13: Pérdida y precisión por época de la red inicial en entrenamiento y validación. La línea punteada marca la época de menor pérdida en validación.

Pruebas para reducir el sobreajuste. Probé varios cambios, uno a la vez, con una muestra de 250 mil posiciones de entrenamiento y 100 mil de validación: *Dropout* con tasas de 0.3 a 0.7, antes y después de la primera capa densa; normalización por lotes (*batch normalization*); una tasa de aprendizaje menor; lotes de 256 y de 1,024; y redes más pequeñas. Solo mejoró el *Dropout* después de la primera capa densa: con una tasa de 0.6, el *top-1* en validación subió de 29.8 % a 31.9 %, y el sobreajuste empezó en la séptima época en lugar de la tercera. Con *Dropout*, agregar capacidad también ayudó: con el doble de filtros en las convoluciones el acierto llegó a 32.1 %, y con una primera capa densa de 2,048 neuronas la pérdida en validación bajó, aunque el acierto no subió. Los demás cambios movieron el acierto menos que la diferencia entre dos semillas (0.2 puntos).

Todas las partidas y la GPU. Después pasé de la muestra a todas las partidas filtradas (14.4 millones de posiciones de entrenamiento y 1.8 millones de validación). Para que cada entrenamiento cupiera en unas cuantas horas, lo hice en la GPU de mi computadora (Apple M5): al repetir el entrenamiento de la red inicial ahí, cada época tardó unos 5 minutos en lugar de 12, con la misma mejor época y prácticamente el mismo acierto en validación (39.1 % contra 39.2 %).

Tres configuraciones con todos los datos. Entrené las tres configuraciones que mejor resultaron con la muestra: *Dropout* de 0.6; *Dropout* de 0.5 con 128 y 256 filtros; y *Dropout* de 0.5 con una primera capa densa de 2,048 neuronas. Cada una tardó entre 2.7 y 3.7 horas, con *EarlyStopping* y como máximo 10, 6 y 7 épocas. Ninguna sobreajustó: en las tres la menor pérdida en validación llegó en la sexta época, y después casi no cambió. El acierto *top-1* en validación fue de 41.3 %, 42.3 % y 43.1 %, y la tercera fue la mejor en todas las fases, la que elegí. Con la muestra, la capa densa ancha era la que menos aportaba; con 58 veces más posiciones de entrenamiento aprovecha sus parámetros sin memorizarlos.

7.3. Entrenamiento final

Con la configuración elegida entrené un modelo nuevo con entrenamiento y validación juntos (16.2 millones de posiciones), durante 6 épocas, las de su menor pérdida en validación en la prueba anterior. Como ya no hay validación que vigilar, el número de épocas quedó fijo de antemano. Tardó 205 minutos en la GPU; la pérdida bajó de 2.45

a 2.01 y el acierto en entrenamiento subió de 34.4 % a 41.0 % (figura 14).

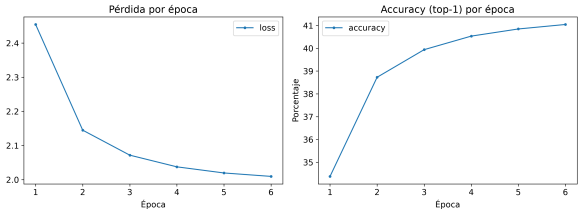


Figura 14: Pérdida y precisión por época del modelo final, entrenado con entrenamiento y validación juntos.

7.4. Resultados en prueba

Este modelo es el único que evalué en el conjunto de prueba, y solo una vez. Acierta la jugada del jugador en el 43.0 % de las posiciones y la tiene entre sus cinco primeras opciones en el 81.1 % (tabla 7). Es casi lo mismo que la configuración elegida acertó en validación (43.1 %).

Para compararlo calculé las líneas base en las posiciones de prueba, con el «libro» y la jugada más frecuente sacados de entrenamiento y validación, los mismos datos que vio el modelo. Supera a todas en todas las fases: en la apertura supera al «libro», más allá de recordar aperturas, y en el medio juego y en el final duplica a la captura de mayor valor.

Antes de descartar las jugadas ilegales, solo el 1.4 % de sus primeras opciones es ilegal, contra el 2.6 % de la red inicial. Esos errores se concentran en el final (3.8 %), la fase con menos posiciones para aprender.

Corte	*Top-1*	*Top-3*	*Top-5*	Línea base	Ilegal
Todas	43.0	70.3	81.1	20.3	1.4
Apertura	53.9	82.7	91.9	47.3	0.1
Medio juego	37.9	64.2	75.6	17.6	1.7
Final	40.1	68.5	80.4	20.6	3.8

Cuadro 7: Acierto del modelo en prueba (%), frente al *top-1* de la mejor línea base de cada corte en las mismas posiciones («libro» en todas y en la apertura; captura de mayor valor en el medio juego y en el final). «Ilegal»: primeras opciones ilegales antes de descartarlas.

7.5. Fuerza de juego

Para estimar el nivel del modelo lo hice jugar contra una escalera de rivales:

- una jugada legal al azar;
- una regla que siempre hace la captura de mayor valor, la misma línea base de antes, ahora como jugador;
- Stockfish limitado a buscar a una sola jugada de profundidad;
- las redes Maia de 1100, 1500 y 1900, que imitan a jugadores de esos niveles y juegan sin buscar;
- Stockfish con su fuerza limitada a 1320, el nivel más bajo que permite.

Cada pareja jugó desde 20 aperturas, una vez con cada color. Cada apertura es la secuencia de 8 *plies* más frecuente de una de las 20 familias más jugadas. El modelo jugó 10 partidas contra cada rival y completó 40 contra los que le hicieron entre el 10 % y el 90 % de los puntos. La tabla 8 muestra el resultado contra cada uno, con las tablas contadas como medio punto.

Rival	Partidas	Puntos (%)
Azar	40	82
Captura	40	71
Stockfish 1320	40	31
Stockfish prof. 1	40	21
Maia 1100	40	15
Maia 1900	40	14
Maia 1500	10	5

Cuadro 8: Resultado del modelo contra cada rival: partidas jugadas y porcentaje de los puntos que hizo el modelo. «Captura»: captura de mayor valor; «prof. 1»: profundidad 1.

El modelo le gana con claridad al azar y a la regla de capturas, y pierde contra Maia y contra Stockfish; su mejor resultado entre estos últimos es contra Stockfish con la fuerza limitada a 1320.

7.6. Análisis de errores

Para ver en qué falla el modelo, analicé con Stockfish, a 10 jugadas de profundidad, las 8,554 jugadas que hizo en sus 250 partidas, sin contar las de la apertura fija. En cada jugada medí cuánto empeora la posición frente a la mejor jugada, en centipeones (centésimas del valor de un peón); si tenía un mate en una jugada y lo dio; y qué probabilidad les asignó a las jugadas ilegales antes de descartarlas (tabla 9).

Fase	Pérdida	Graves	Ilegal	Mates
Apertura	43	2.9	0.3	1/2
Medio juego	98	9.2	5.8	43/357
Final	54	3.6	16.3	8/64
Todas	80	7.1	6.8	52/423

Cuadro 9: Jugadas del modelo en las partidas de la escalera, analizadas con Stockfish (1,497 jugadas en la apertura, 5,511 en el medio juego y 1,546 en el final). «Pérdida»: pérdida media en centipeones; «graves»: jugadas que pierden 300 centipeones o más, más o menos una pieza menor (%); «ilegal»: probabilidad media que el modelo asignó a jugadas ilegales (%); «mates»: veces que dio mate entre las que tenía un mate en una.

Rematar las partidas ganadas. Es la falla más clara. En 423 jugadas tenía un mate en una y solo lo dio en 52 (12 %); en 415 de esas jugadas llevaba al menos 5 puntos de material de ventaja. Con esa ventaja, 42 partidas terminaron en tablas: 18 por ahogado y 24 por repetición. Hay dos causas probables. Primero, entre jugadores de 2,200 o más quien va muy abajo suele abandonar: en los datos casi no hay posiciones con tanta ventaja ni jugadas de mate en ellas. Segundo, el modelo solo ve la posición actual y no sabe que la está repitiendo.

Errores tácticos en el medio juego. En el medio juego el 9.2 % de sus jugadas pierde al menos una pieza menor, contra el 2.9 % en la apertura y el 3.6 % en el final. Es también la fase en la que menos acierta en prueba (37.9 %). El modelo elige la jugada que se parece a la de una persona, pero no calcula qué pasa después. Además, sus dos capas convolucionales solo combinan zonas de 5×5 casillas; las relaciones a lo largo de todo el tablero, como una dama o un alfil que ataca desde lejos, quedan a cargo de las capas densas.

Posiciones poco comunes. La probabilidad que asigna a jugadas ilegales sube del 0.3 % en la apertura al 16.3 % en el final, y del 2.4 % en las posiciones con material parejo al 15.2 % cuando va 5 puntos o más abajo. En las posiciones que se parecen poco a las de entrenamiento todavía duda de cómo se mueven las piezas.

7.7. Uso del modelo

El modelo se puede usar de dos maneras. En la ventana de juego se elige quién lleva cada color (una persona, el modelo, Stockfish o un jugador al azar) y se juega con el ratón; cuando mueve el modelo, la ventana muestra sus tres jugadas más probables con una flecha y una barra del mismo color, la probabilidad que puso en jugadas ilegales y, junto al tablero, la ventaja de la posición según Stockfish (figura 15). Por consola, el modelo recibe una posición escrita en FEN y devuelve sus cinco jugadas más probables con su probabilidad.

8. Códigos

Todos los códigos están en https://github.com/TheSimpleMau/Portafolio-Implementacion-Deeplearning. El archivo README explica cómo ejecutarlos.



Figura 15: Ventana del juego tras una jugada del modelo. Las flechas y las barras del panel muestran sus tres jugadas más probables; la barra de la izquierda, la ventaja según Stockfish.

Referencias

[1] Muhammad Faiz Arsalan y Haryono Soeparno. "Deep Learning Approaches to Predicting the Optimal Chess Moves from Board Positions". En: *International Journal of Engineering Trends and Technology* 72.4 (2024), págs. 43-50. DOI: 10.14445/22315381/IJETT-V72I4P105.

[2] Timo Bertram, Johannes Fürnkranz y Martin Müller. "Neural Network-Based Information Set Weighting for Playing Reconnaissance Blind Chess". En: *IEEE Transactions on Games* 16 (2024), págs. 960-970. DOI: 10.1109/TG.2024.3425803.

[3] Niklas Fiekas. *python-chess: a chess library for Python*. https://python-chess.readthedocs.io/. Versión 1.11.2. [Consultado 15-10-2026].

[4] Lichess. *Lichess Open Database*. https://database.lichess.org/. Licencia CC0. [Consultado 15-10-2026].

[5] Reid McIlroy-Young et al. "Aligning Superhuman AI with Human Behavior: Chess as a Model System". En: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '20. ACM, 2020, págs. 1677-1687. DOI: 10.1145/3394486.3403219.

[6] Heigon Soldera y Helio Pedrini. "Chess Board Digitization and Human Move Prediction Using Machine Learning". En: *IEEE Access* 14 (2026), págs. 81589-81615. DOI: 10.1109/ACCESS.2026.3697295.